

Nuestra caja de herramientas

El Entorno

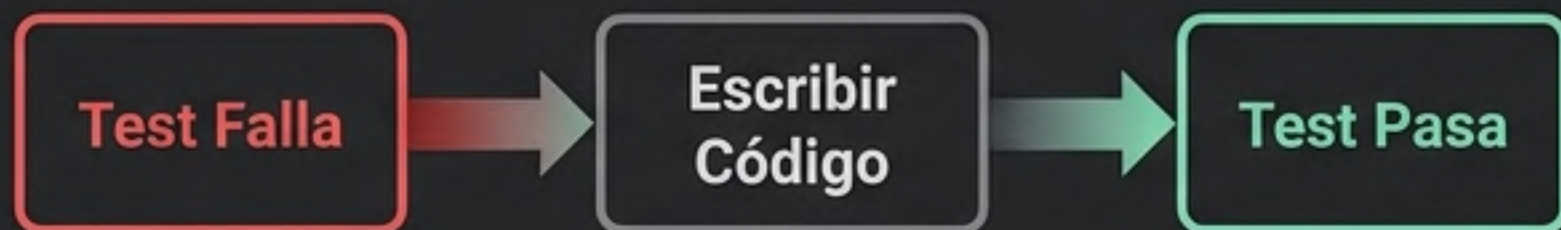


Vitest: El Test Runner. Ejecuta pruebas a la velocidad de la luz.



Vue Test Utils (VTU): Librería para montar e interactuar con componentes Vue aislados.

El Ciclo de Vida (TDD)



Cheat Sheet Rápido

- 1 `mount()` : Crea el wrapper del componente.
 - 2 `find('[data-testid=...]')` : Busca un elemento en el DOM simulado.
 - 3 `setValue()` : Escribe dentro de un input.
 - 4 `trigger()` : Simula eventos de usuario (clics, teclado).
 - 5 `await` : Espera que Vue actualice el DOM asincrónicamente.
- >_ █

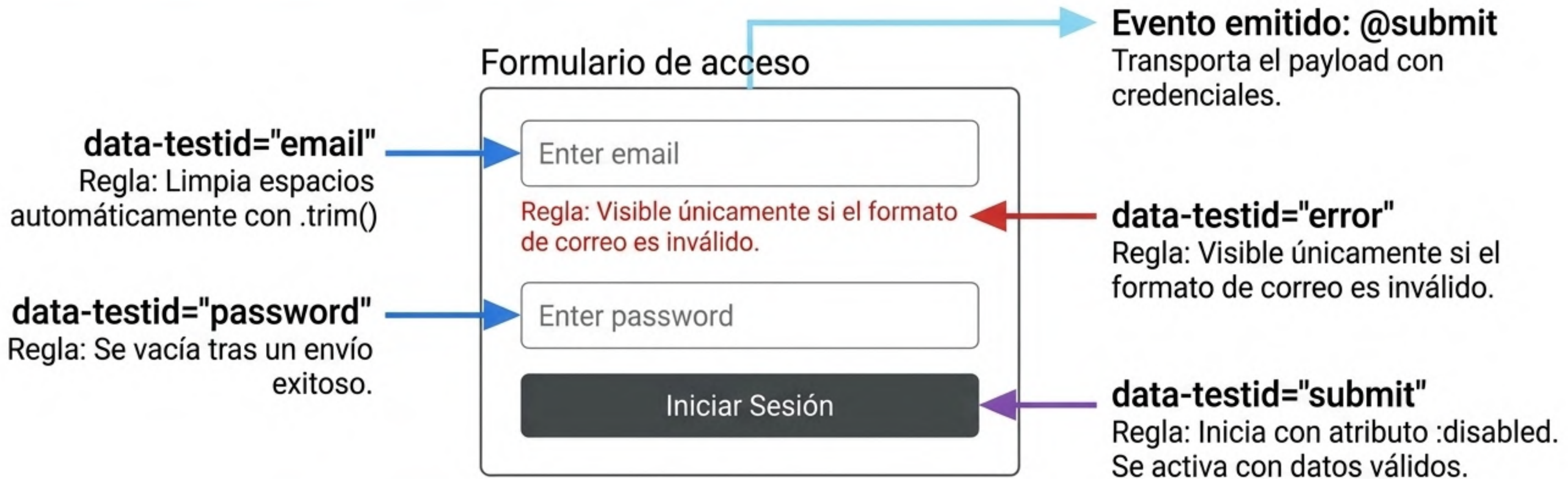
Llevando los tests al mundo real

>_ Ejecutando agenda_clase_2.sh

1. Formularios completos: Interacción con `inputs`, `validaciones` reactivas y `estados` de botones.
2. Emisión de `eventos (emitted)`: Comprobar la comunicación hacia el exterior.
3. Dependencias falsas: Introducción visual a `Mocks` y `Stubs`.
4. Preparación táctica: Desglose paso a paso del ejercicio `LoginForm.vue`.

Status: Ready

La anatomía del objetivo: un formulario testeable



El mismo patrón se puede aplicar a LoginForm.vue, AccessForm.vue u otros formularios simples.

Nuestro plan de pruebas



✓ LoginForm.spec.ts

- ✓ Test 1: Renderiza correctamente el estado inicial.
 - Inputs existentes.
 - Inputs vacíos.
 - Botón inactivo.
 - Sin error inicial.
- ✓ Test 2: Habilita el botón al recibir datos válidos.
- ✓ Test 3: Muestra mensaje de error ante email inválido.
- ✓ Test 4: Emite el evento submit con el payload correcto.
- ✓ Test 5: Limpia el campo password/código tras un envío exitoso.

El enfoque TDD nos exige escribir cada uno de estos tests ANTES de implementar la lógica del componente.

El código base del componente

```
<script setup>
import { ref, computed } from 'vue'

const email = ref('')
const password = ref('')

const emit = defineEmits(['submit'])

const isValidEmail = computed(() =>
  /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email.value.trim())
)

const canSubmit = computed(() =>
  isValidEmail.value && password.value.length > 0
)

function handleSubmit() {
  if (!canSubmit.value) return

  emit('submit', {
    email: email.value.trim(),
    password: password.value
  })

  password.value = ''
}
</script>
```

```
<template>
  <form data-testid="form" @submit.prevent="handleSubmit">
    <input data-testid="email" v-model="email" />
    <input data-testid="password" v-model="password"
      type="password" />

    <p v-if="email && !isValidEmail" data-testid="error">
      Email inválido
    </p>

    <button data-testid="submit" :disabled="!canSubmit">
      Ingresar
    </button>
  </form>
</template>
```

Test 1: Verificando el estado por defecto

Concepto

Objetivo

Asegurar que el DOM se construye exactamente como esperamos al iniciar, protegiendo al usuario de clics accidentales.

Tip Táctico

Nunca uses selectores CSS frágiles como **.btn-primary**. Usa siempre **data-testid** para que tus tests sobrevivan a rediseños visuales.

```
it('renderiza inputs y botón deshabilitado', ()
=> {
  const wrapper = mount(LoginForm)

  // Los inputs deben existir
  expect(wrapper.find('[data-testid="email"]')
.exists()).toBe(true)

  // El botón debe estar deshabilitado
  const submitBtn = wrapper.find('[data-
testid="submit"]')
  expect(submitBtn.attributes('disabled'))
.toBeDefined()

  // El error NO debe existir inicialmente
  const errorMsg = wrapper.find('[data-
testid="error"]')
  expect(errorMsg.exists()).toBe(false)
})
```

Test 2: Escribiendo en el formulario

Objetivo

Simular un usuario llenando el formulario y comprobar que el botón reacciona habilitándose.

¿Por qué usamos await?

Vue actualiza el DOM de forma asincrónica. Si modificamos un valor con `setValue`, debemos decirle al test que **ESPERE** a que Vue re-renderice la pantalla antes de verificar el estado del botón.

```
it('habilita el botón con datos válidos', async () => {
  const wrapper = mount(LoginForm)

  await wrapper.find('[data-testid=email]')
    .setValue('user@vue.com')

  await wrapper.find('[data-testid=password]')
    .setValue('secreta123')

  // El atributo ya no debe existir
  const submitBtn = wrapper.find('[data-testid=submit]')
  expect(submitBtn.attributes('disabled')).toBeUndefined()
})
```

Test 3: El camino triste y validación reactiva

Objetivo

Objetivo

Comprobar que nuestra lógica de validación protege el envío y advierte al usuario de un error de formato.



Visibilidad Condicional

Si el mensaje se renderiza con **v-if**, usamos **exists()**. Si el mensaje se oculta con **v-show**, el elemento puede existir en el DOM, pero no ser visible; en ese caso usamos **isVisible()**.

```
it('muestra error si el email es inválido', async () => {
  const wrapper = mount(LoginForm)

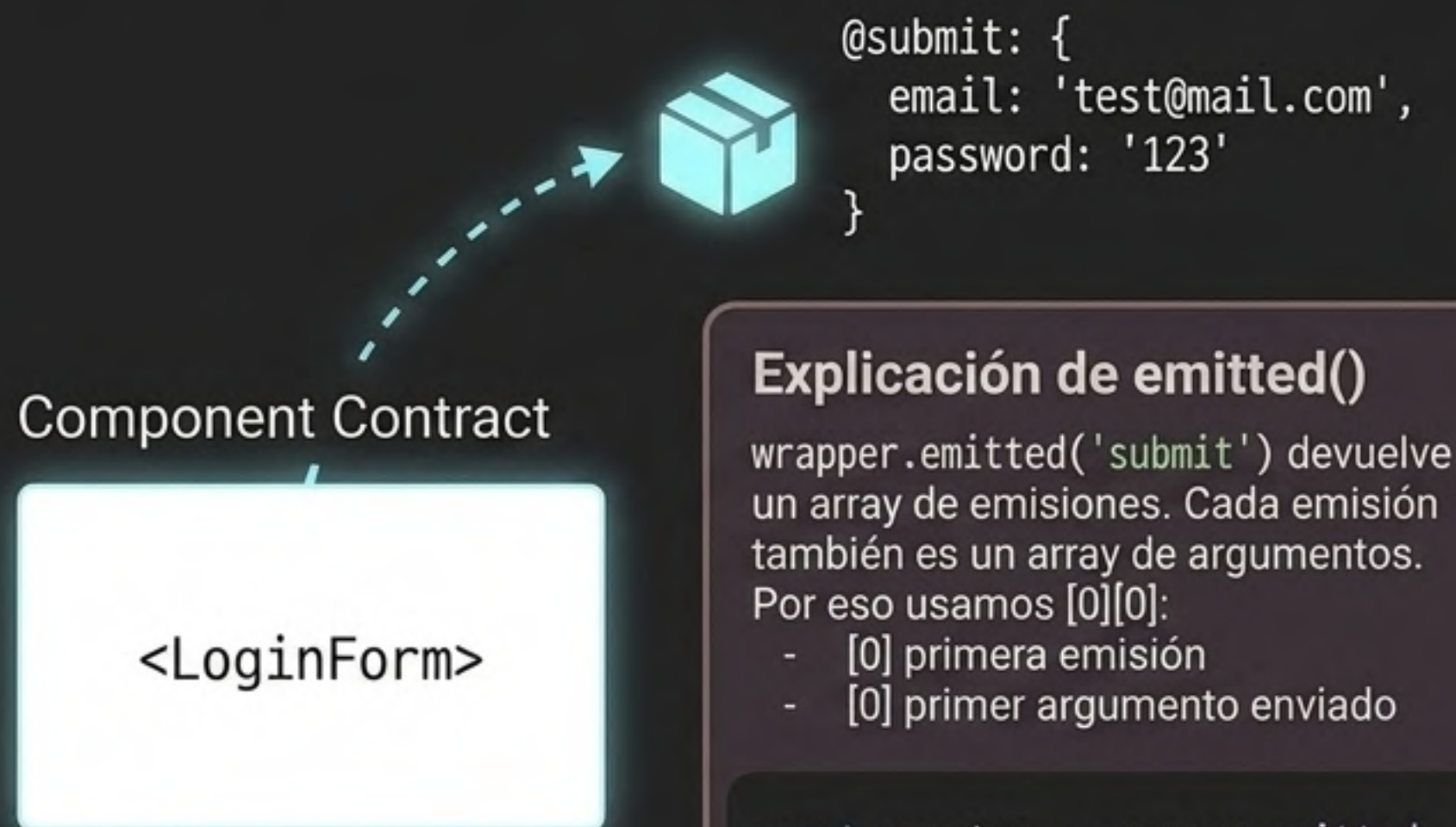
  // Ingresamos un email sin arroba
  await wrapper.find('[data-testid=email]')
    .setValue('usuario-sin-arroba.com')

  await wrapper.find('[data-testid=password]')
    .setValue('1234')

  // Verificamos que el mensaje aparece
  const error = wrapper.find('[data-testid="error"]')
  expect(error.exists()).toBe(true)
  expect(error.text()).toContain('Email inválido')

  // El botón debe seguir protegido
  const submitBtn = wrapper.find('[data-testid=submit]')
  expect(submitBtn.attributes('disabled')).toBeDefined()
})
```

Test 4: Eventos emitidos y el payload



Explicación de emitted()

`wrapper.emitted('submit')` devuelve un array de emisiones. Cada emisión también es un array de argumentos. Por eso usamos `[0][0]`:

- `[0]` primera emisión
- `[0]` primer argumento enviado

```
const eventos = wrapper.emitted('submit')
expect(eventos).toHaveLength(1)
expect(eventos[0][0]).toEqual({
  email: 'test@mail.com',
  password: '123'
})
```

```
// 1. Disparamos el evento nativo del formulario
await wrapper.find('form').trigger('submit.prevent')

// 2. Comprobamos que el evento ocurrió
expect(wrapper.emitted('submit')).toBeTruthy()

// Verificación detallada con el payload (ver explicación a la izquierda)
const eventos = wrapper.emitted('submit')

expect(eventos).toHaveLength(1)
expect(eventos[0][0]).toEqual({
  email: 'test@mail.com',
  password: '123'
})
```

Test 5: Limpiando la casa post-envío

Objetivo

Objetivo

Mejorar la experiencia de usuario y la seguridad. Después de un intento de login, borramos la contraseña de la pantalla instantáneamente, pero conservamos el email para que el usuario no deba teclearlo de nuevo.

Validación de Elementos

Probaremos directamente el valor contenido en el elemento HTML del DOM simulado.

```
it('limpia el password pero mantiene el email', async
  () => {

    const emailInput = wrapper.find('[data-
      testid="email"]')
    const passwordInput = wrapper.find('[data-
      testid="password"]')

    await emailInput.setValue('test@mail.com')
    await passwordInput.setValue('123')
    await wrapper.find('[data-testid="form"]').trigger(
      ('submit.prevent'))

    expect(passwordInput.element.value).toBe('')
    expect(emailInput.element.value).toBe('test@mail.com')
  })
```

Aislado y controlado: Mocks y Stubs

El Mock

El Doble de Acción

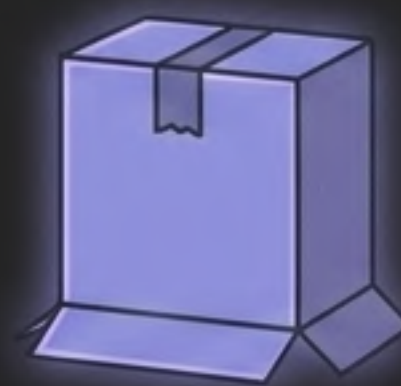


Una dependencia falsa que nosotros controlamos y espiamos.

Uso: Reemplazar llamadas a APIs reales, librerías externas complejas o validadores pesados. Obliga al código a pensar que la red o la base de datos funcionó perfectamente.

El Stub

La Silueta de Cartón



Un reemplazo vacío y pasivo para un componente hijo.

Uso: Cuando el componente padre renderiza un hijo gigante o pesado (ej. un mapa interactivo), lo reemplazamos por un div vacío. No nos importa qué hace el hijo, solo probar al padre aisladamente.

Nota: Para el Loginform de hoy no son obligatorios, pero es vital conocerlos para el ecosistema Vue.

Opcional: mockear un helper de validación

```
vi.mock('@utils/validations', () => ({  
  isValidEmail: vi.fn(() => true)  
}))
```

Para el ejercicio **no es obligatorio** usar mocks. Solo tendría sentido si extraemos la validación a un archivo externo, por ejemplo **utils/validations.js**.

El panorama: Unit Testing vs. E2E

	Unit Testing	E2E Testing
Alcance	Prueba unidades aisladas (un solo componente a la vez).	Prueba flujos de usuario completos (App viva).
Velocidad	Ultrarrápido (miles de tests en segundos).	Lento (minutos).
Entorno	Simulado en memoria (JSDOM / Happy DOM).	Navegador web real conectado a la base de datos.
Herramientas	Vitest + Vue Test Utils	Cypress, Playwright, Nightwatch

Nuestro ejercicio de hoy es estrictamente Unitario.

Tu plan de vuelo para el ejercicio

1. Setup Inicial

Crea LoginForm.vue y LoginForm.spec.ts.

3. TDD en acción

Escribe el Test 1 (que falle) y agrega la lógica mínima para el Verde.

5. El contrato

Prueba la emisión del evento @submit con el payload.

¡Ejecuta `pnpm test:unit` y consigue todos los checks en verde!

```
> pnpm test:unit  
> pnpm test:unit --run
```

2. Esqueleto

Diseña el <template> y agrega los atributos data-testid.

4. Interacción

Resuelve las validaciones interactuando con los inputs.

6. Pulido de UX

Prueba la limpieza del campo password.

La regla de oro del testing

No pruebes detalles de implementación.

(No verifiques el valor interno de variables reactivas ni las clases CSS específicas).



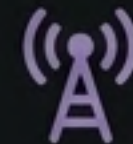
Contratos observables
Lo que el usuario VE

El DOM renderizado, mensajes de error, botones activos.



Contratos observables
Lo que el usuario HACE

Clics en botones, tipeo en inputs.



Contratos observables
Lo que el componente EMITE

Eventos (@submit) y payloads enviados a su padre.

¡Estás listo para el laboratorio. A codificar!